

---

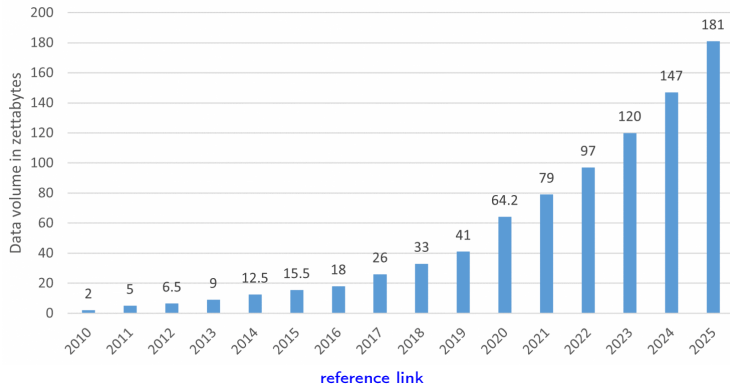
# Special Topics in AI

## Lecture 7: Introduction to Federated Learning

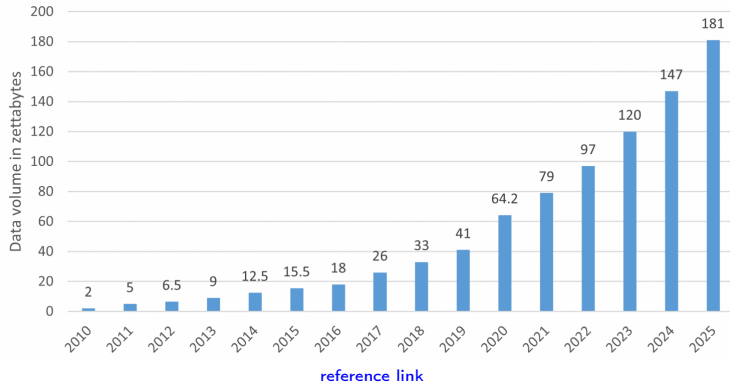
University of Mazandaran

Fall 1405

# The Data Boom



# The Data Boom



FYI, 1 Zettabytes =  $1e + 9$  Terabytes!

# Data is Born At the Edge

Billions of phones and IoT devices constantly generate data



# Data is Born At the Edge

Billions of phones and IoT devices constantly generate data

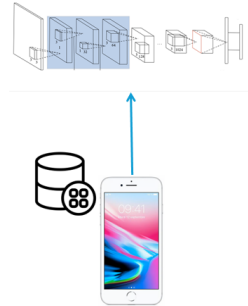
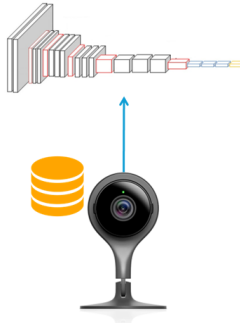
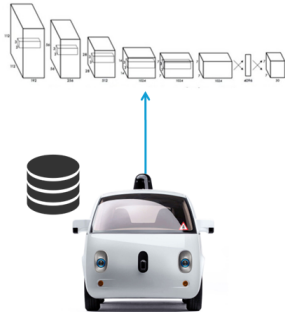
More data enable better and smarter models



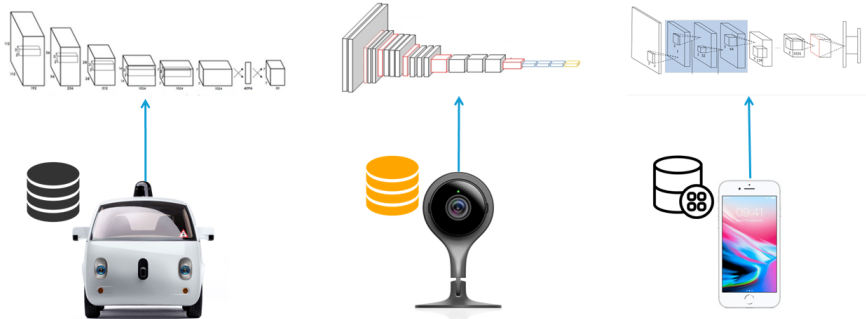
# Can Data Live at the Edge?



# Can Data Live at the Edge?

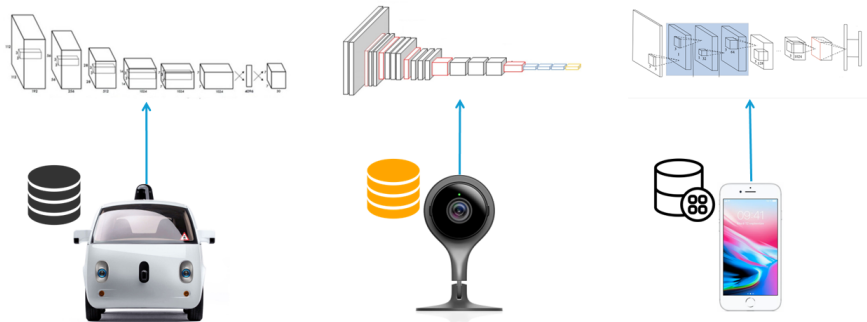


# Can Data Live at the Edge?



Not yet!

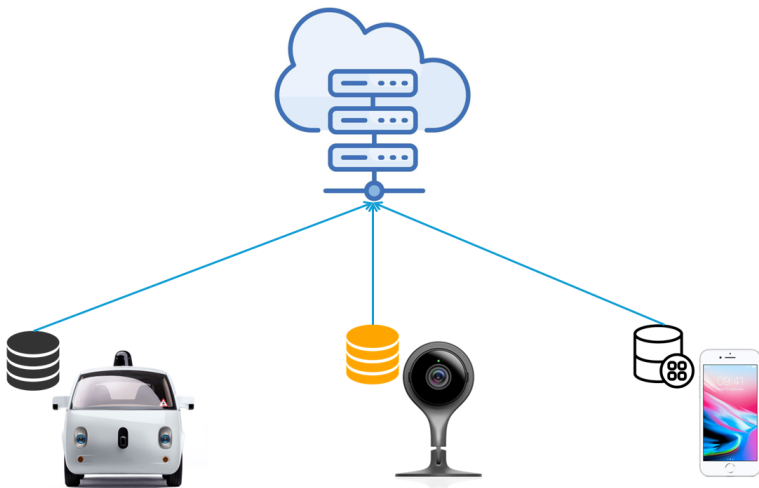
# Can Data Live at the Edge?



Not yet!

- ▶ resource constraints

# Centralized Learning



# Challenges of Centralized Learning

---

- ▶ Sharing data with the central node may be too costly
  - ▶ bandwidth constraints for wireless devices
- ▶ Data may be considered too sensitive
  - ▶ data privacy in business

# Paradigm Shift: From Centralized to Decentralized

divide the large-scale learning process on one workstation into several small learning processes on a number of distributed workstations



A decentralized learning framework in which a server and several clients collaboratively train a ML model, without sharing data

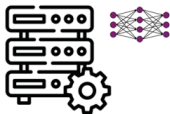
A decentralized learning framework in which a server and several clients collaboratively train a ML model, without sharing data

- ▶ A data privacy-aware distributed learning framework

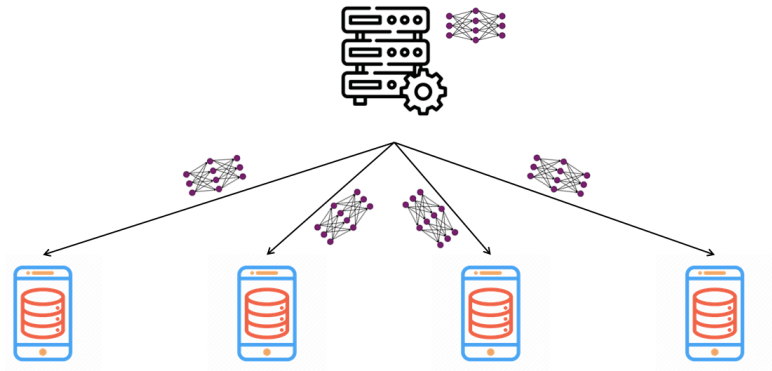
# Federated Learning



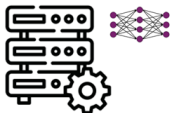
## 1. Initialize model



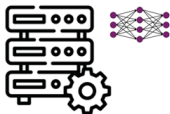
## 1. Initialize model



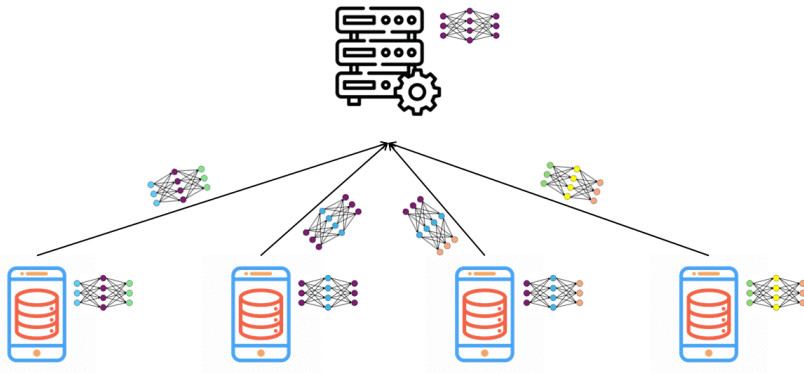
# Federated Learning



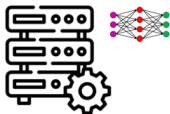
2. Each client makes an update using its local dataset



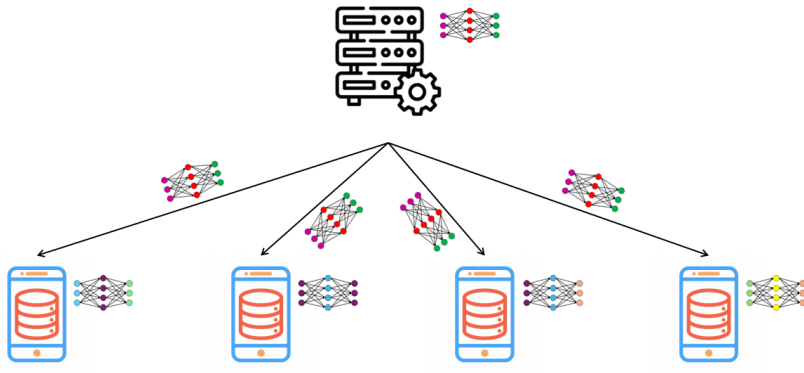
## 3. Clients share their local updates with the server



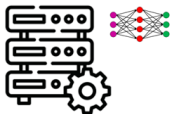
## 4. Server aggregates the updates



## 5. Server shares the aggregated model with the clients



Repeat 1-5 until converges



# Federated Learning: Defining Characteristics:

---

- ▶ Data is generated locally and remains decentralized
- ▶ Clients cannot see each other's data
- ▶ Server cannot read clients' data
- ▶ Data may not be I.I.D.

# Key Design Questions

---

- ▶ How to aggregate client models?
- ▶ What loss function should be used in the clients?
- ▶ What optimization algorithm should be used in the clients?
- ▶ What should be shared between clients and server?
- ▶ Which clients should participate in each round?
- ▶ How much local computation should clients perform before communicating with the server?
- ▶ etc.

# Baseline Algorithm: FedAvg

Assume set of  $K$  clients, each holds a dataset  $\mathcal{D}_k$  of  $n_k$  samples

- ▶  $\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_K$

- ▶  $n = \sum_k n_k$

We want to solve a problem of the form  $\min_{\theta \in \mathbb{R}^p} F(\theta; \mathcal{D})$  where:

$$F(\theta; \mathcal{D}) = \sum_{k=1}^K \frac{n_k}{n} F_k(\theta; \mathcal{D}_k)$$

$$F_k(\theta; \mathcal{D}_k) = \sum_{d \in \mathcal{D}_k} f(\theta; d)$$

$\theta \in \mathbb{R}^p$  are model parameters

# Baseline Algorithm: FedAvg

---

## Server executes:

initialize  $w_0$

**for** each round  $t = 1, 2, \dots$  **do**

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$  (random set of  $m$  clients)

**for** each client  $k \in S_t$  **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$m_t \leftarrow \sum_{k \in S_t} n_k$

$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$

**ClientUpdate**( $k, w$ ): *// Run on client  $k$*

$\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )

**for** each local epoch  $i$  from 1 to  $E$  **do**

**for** batch  $b \in \mathcal{B}$  **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

return  $w$  to server

---

McMahan et al. "Communication Efficient Learning of Deep Networks from Decentralized Data"

# Baseline Algorithm: FedAvg

**Server executes:**

initialize  $w_0$

**for** each round  $t = 1, 2, \dots$  **do**

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$  (random set of  $m$  clients)

← client selection: random

**for** each client  $k \in S_t$  **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$m_t \leftarrow \sum_{k \in S_t} n_k$

$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$

← aggregation: averaging

**ClientUpdate**( $k, w$ ): // Run on client  $k$

$\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )

**for** each local epoch  $i$  from 1 to  $E$  **do**

**for** batch  $b \in \mathcal{B}$  **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

← optimization: GD

return  $w$  to server

McMahan et al. "Communication Efficient Learning of Deep Networks from Decentralized Data"

# Baseline Algorithm: FedAvg

---

**Server executes:**

initialize  $w_0$

**for** each round  $t = 1, 2, \dots$  **do**

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$  (random set of  $m$  clients)

**for** each client  $k \in S_t$  **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$m_t \leftarrow \sum_{k \in S_t} n_k$

$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$

**ClientUpdate**( $k, w$ ): *// Run on client  $k$*

$\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )

**for** each local epoch  $i$  from 1 to  $E$  **do**

**for** batch  $b \in \mathcal{B}$  **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

return  $w$  to server

---

McMahan et al. "Communication Efficient Learning of Deep Networks from Decentralized Data"

What if  $E = 1$  and  $C = 1$ ?

# Baseline Algorithm: FedAvg

**Server executes:**

initialize  $w_0$

**for** each round  $t = 1, 2, \dots$  **do**

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$  (random set of  $m$  clients)

**for** each client  $k \in S_t$  **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$m_t \leftarrow \sum_{k \in S_t} n_k$

$w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$

**ClientUpdate**( $k, w$ ): *// Run on client  $k$*

$\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )

**for** each local epoch  $i$  from 1 to  $E$  **do**

**for** batch  $b \in \mathcal{B}$  **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

return  $w$  to server

---

McMahan et al. "Communication Efficient Learning of Deep Networks from Decentralized Data"

What if  $E = 1$  and  $C = 1$ ?

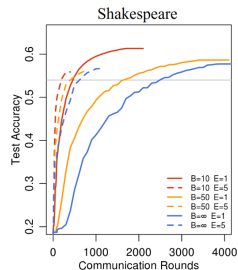
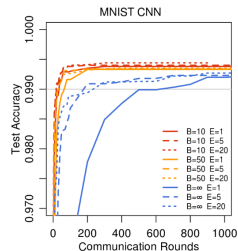
► FedAvg becomes parallel SGD

# Baseline Algorithm: FedAvg

A CNN with two  $5 \times 5$  convolution layers, a fully connected layer with 512 units and ReLu activation, and a final softmax output layer is trained considering in total 100 clients each receiving 600 examples.

A stacked character-level LSTM is trained with 1146 clients.

In both experiments,  $C = 0.1$ . The gray lines show the target accuracies.



## A Minimal Experiment

We will reproduce the FedAvg loop with only a few lines of PyTorch code.

- ▶ Dataset: `sklearn` handwritten digits (1797 images,  $8 \times 8$  pixels)
- ▶ 5 clients, each holding a disjoint IID subset of the training data
- ▶ Model: small MLP,  $64 \rightarrow 32 \rightarrow 10$
- ▶ Each client trains for  $E = 1$  local epoch in every communication round
- ▶ Server uses weighted FedAvg:  $\theta \leftarrow \sum_k \frac{n_k}{n} \theta_k$

**Goal:** watch global test accuracy change after each communication round.

## Step 1 – Create the Clients

```
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype("float32")

Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=7)

train_set = TensorDataset(torch.tensor(Xtr), torch.tensor(ytr))

# IID partition among 5 clients
idx = np.random.permutation(len(train_set))
client_idx = np.array_split(idx, 5)
client_loaders = [
    DataLoader(Subset(train_set, ids.tolist()),
                batch_size=32, shuffle=True)
    for ids in client_idx
]
client_sizes = [len(ids) for ids in client_idx]
```

## Step 2 – Define One Shared Model

```
class SmallMLP(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(64, 32),  
            nn.ReLU(),  
            nn.Linear(32, 10)  
        )  
  
    def forward(self, x):  
        return self.net(x)
```

- ▶ The server initializes `global_model = SmallMLP()`.
- ▶ Every selected client starts each round from the **same global parameters**.
- ▶ Only model parameters are exchanged; raw images stay local.

## Step 3 – Local Training at One Client

```
def local_train(global_model, loader, lr=0.1):  
    model = copy.deepcopy(global_model)  
    opt = torch.optim.SGD(model.parameters(), lr=lr)  
    loss_fn = nn.CrossEntropyLoss()  
  
    model.train()  
    for xb, yb in loader:                # one local epoch  
        opt.zero_grad()  
        loss = loss_fn(model(xb), yb)  
        loss.backward()  
        opt.step()  
  
    return model.state_dict()           # send parameters, not data
```

The same function runs independently on Client 1, Client 2, ..., Client 5.

## Step 4 – FedAvg at the Server

$$\theta^{t+1} = \sum_{k=1}^K \frac{n_k}{\sum_j n_j} \theta_k^{t+1}$$

```
def fedavg(client_states, client_sizes):  
    total = sum(client_sizes)  
    avg = copy.deepcopy(client_states[0])  
  
    for key in avg:  
        avg[key] = sum(  
            state[key] * (n_k / total)  
            for state, n_k in zip(client_states,  
                                  client_sizes)  
        )  
    return avg
```

## Step 5 – The Federated Training Loop

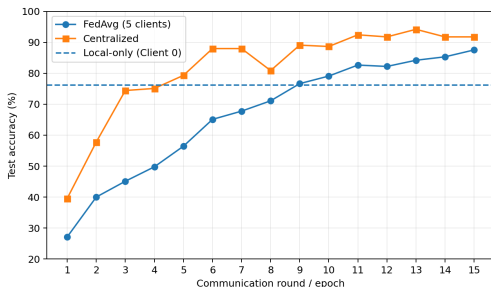
```
global_model = SmallMLP()

for rnd in range(1, 16):
    # 1. broadcast global model + 2. local training
    client_states = [
        local_train(global_model, loader)
        for loader in client_loaders
    ]

    # 3. aggregate local models
    new_state = fedavg(client_states, client_sizes)
    global_model.load_state_dict(new_state)

    # 4. evaluate the new global model
    acc = accuracy(global_model, test_loader)
    print(f"Round {rnd:2d}: accuracy = {acc:.3f}")
```

## What Does FedAvg Achieve?



After 15 rounds in this run:

- ▶ Local only: **76.2%**
- ▶ FedAvg: **87.6%**
- ▶ Centralized: **91.8%**

FedAvg learns from **all five clients** without collecting their raw data.

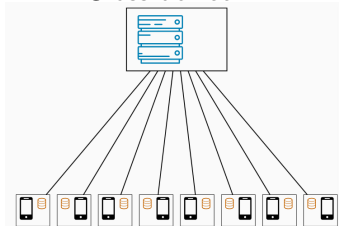
## Reading the Result

- ▶ Accuracy generally increases as more communication rounds are completed.
- ▶ The gap depends on learning rate, local epochs  $E$ , client sampling, and especially data heterogeneity.
- ▶ This example is intentionally IID and small. It is the **clean baseline** before studying non-IID FL.

Next experiment: give different clients different digit classes. What do you expect to happen?

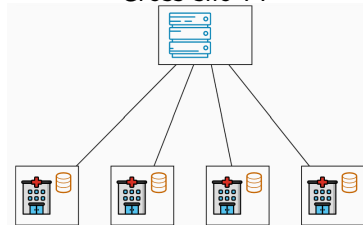
# Cross-Device vs. Cross-Silo FL

Cross-device FL



- ▶ Massive number of clients (up to  $10^{10}$ )
- ▶ Small dataset per party
- ▶ Limited availability and reliability
- ▶ Some clients may be malicious

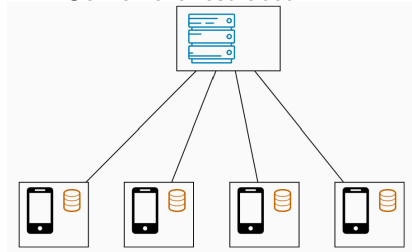
Cross-silo FL



- ▶ Much smaller number of clients (2 – 100)
- ▶ Medium to large dataset per client
- ▶ Reliable clients, almost always available
- ▶ Clients are typically honest

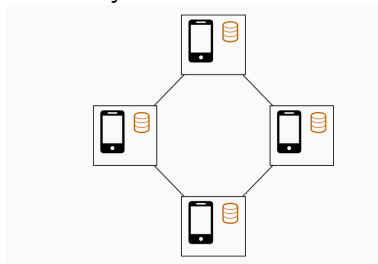
# Server Orchestrated vs. Fully Decentralized

Server-orchestrated FL



- ▶ Server-client communication
- ▶ Global coordination, global aggregation
- ▶ Server is the bottleneck

Fully decentralized FL



- ▶ Device-to-device communication
- ▶ No global coordination, local aggregation
- ▶ Scales to a large number of devices

# Main Challenges and Future Directions

